

Network Anomaly Root-Cause Assistant

The Complete Study Bible

Problem - Solution - Layer-by-layer implementation, in plain first-principles terms

0. The Problem

A network operations centre (NOC) watches hundreds of machines - load balancers, web servers, application servers, databases, caches, switches. When something goes wrong, the monitoring system does not send one alert. It sends a storm of them - often thousands per hour.

Here is the trap. When a database slows down, every service that depends on it also slows down. The web tier times out, the app tier queues up, dashboards turn red everywhere. To the on-call engineer, the web server looks broken - it has the loudest alarms and the worst-looking numbers. But the web server is a victim, not the villain. The real culprit is the database, three hops away, quietly refusing new connections.

Why existing tools get this wrong

Most tools correlate events by time: 'these ten things all went bad around 14:32, so they are related.' That is correlation. It tells you what moved together, not what caused what. Correlation-based tools routinely blame whichever component moved first or looks worst - which is very often the victim. The engineer then spends the bulk of mean-time-to-resolution just figuring out which red light is the cause and which are downstream noise.

The core question this project answers

Given a storm of anomalies across a network, which single component is the ROOT CAUSE - and how do we prove it, rather than guess by timing? The answer is the difference between correlation (things moved together) and causation (this made that happen).

What 'good' looks like

- Point to ONE component as the root cause, with a confidence score.
- Show the exact chain of how the fault propagated (cause -> ... -> symptom).
- Justify it with evidence, separating hard proof from mere coincidence.
- Say what to check next, and prove the whole investigation was not tampered with.
- Be reproducible - the same incident must give the same answer every time.

1. Overview of the Solution

Our system takes in five kinds of network data, finds the anomalies, and then - this is the key idea - uses the network's own DEPENDENCY MAP (its topology) to decide which anomaly caused the others. A component can only be blamed if a real dependency path exists from it to the thing that is visibly failing. Everything else is demoted, and we explain why.

The one big idea: causation through topology

Imagine each machine as a box, and draw an arrow from A to B whenever A depends on B (the web server depends on the app server, which depends on the database). If the web server is the visible symptom, the only things allowed to be its root cause are the ones you can reach by walking backwards along those arrows: app server, then database. A cache that is on fire but is NOT on that path cannot be the cause of THIS symptom - no matter how bad it looks. That single rule is what converts a correlation engine into a causation engine.

The two-layer philosophy: 'vending machine, not slot machine'

A slot machine gives a different, unaccountable answer every pull. A vending machine gives the same, predictable item every time. Root-cause analysis must be a vending machine. So we split the system in two:

- A DETERMINISTIC CORE decides the root cause using math and graph rules. Same input -> same answer, always, and every decision can be traced back to evidence.
- A BOUNDED AI LAYER (the language model) only does language work: writing the timeline, explaining in plain English, adversarially double-checking, and recommending next steps. It NEVER picks the cause, and its influence on the confidence score is clamped so it can never flip the ranking.

Why this matters for a jury

Anyone can wire up 'ask ChatGPT what the root cause is.' That is a slot machine - plausible, unaccountable, different every time. Our differentiator is that the DECISION is deterministic and explainable, and the AI is used only where language genuinely helps. That is 'AI delivered right.'

The end-to-end flow

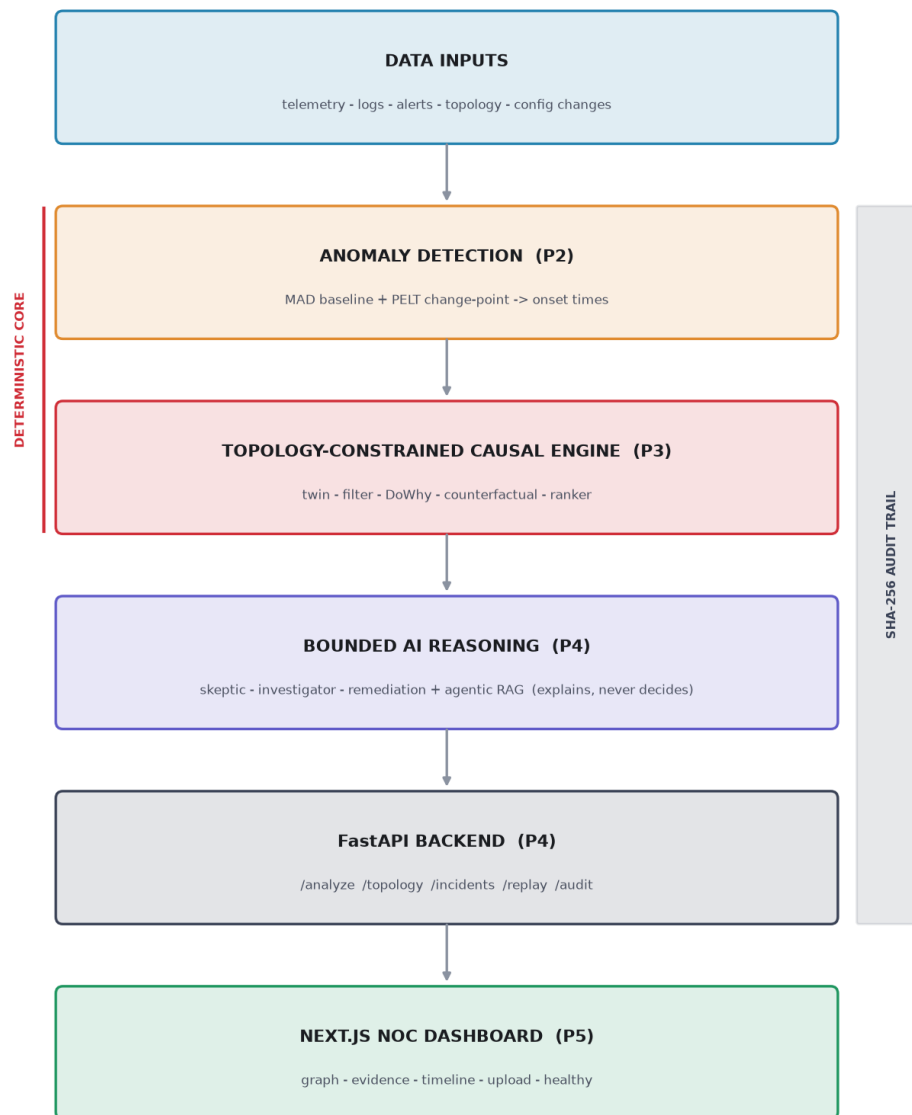


Figure: The full system, top to bottom. Data flows down; the deterministic core (red bracket) decides the cause; the AI layer only explains; every step is written to a hash-chained audit trail.

2. The Layer Map - How the Team Divided the Work

The project is built as six stacked layers, each owned by one role (P0 to P5). The trick that let six people build in parallel without stepping on each other was to FREEZE THE DATA CONTRACT first (Layer 0). Once everyone agreed on the exact shape of the data, each layer could be built and tested against that same shape independently.

Layer	Owner	Responsibility
L0 Data Contract	P0	Define the exact, frozen shape of every data object everyone shares.
L1 Data & Ground Truth	P1	Produce realistic incident data with a known, recorded 'true answer'.
L2 Anomaly Detection	P2	Find anomalies and, crucially, WHEN each one started (onset time).
L3 Causal Engine	P3	The moat: use topology to rank the true root cause and build evidence.
L4 AI + API + Audit	P4	LLM agents that explain/verify; the API; the tamper-proof audit trail.
L5 Frontend	P5	The NOC dashboard: graph, evidence ledger, timeline, upload, healthy view.

Two feature chapters at the end cover things that cut across layers: the SYNTHETIC INCIDENT GENERATOR (how we prove accuracy honestly) and the UPLOAD & HEALTHY-DATA PRODUCT (how a user analyses their own data).

How to read the rest of this document

Every layer chapter follows the same recipe so it reads like a textbook: (1) what it does in one line, (2) first-principles intuition, (3) how it actually works, (4) the key files, (5) what is NOVEL about our approach, and (6) future scope. Read it top to bottom, or jump to the layer you own.

3. Layer 0 - The Data Contract

AUTHOR: P0 - Data Contract & Schema (also owns P3)

KEY FILES:

- contracts/schemas.py (13 frozen Pydantic v2 models + invariants)
- contracts/schemas.json (auto-exported JSON Schema for the frontend)
- contracts/fixtures/ (the golden reference incident: all streams + ground truth)

In one line

The single source of truth for the exact shape of every piece of data the whole system passes around.

First principles: why a contract comes first

Six people cannot build six connected layers at once unless they agree - precisely and in advance - on what the data looks like. If P2 thinks an anomaly has a field called 'start_time' but P3 reads 'onset_ts', nothing connects. A 'data contract' is a written, machine-checked agreement: here is exactly what a Component looks like, a TelemetryPoint, an Anomaly, a Hypothesis, an IncidentReport. Freeze it, and everyone can build against it in parallel and trust that the pieces will click together.

How it works

We use Pydantic v2 models with two strict settings: 'frozen' (once created, an object cannot be secretly modified) and 'extra=forbid' (no stray, undocumented fields can sneak in). Beyond just field types, the models encode INVARIANTS - rules that must always hold - so bad data is rejected at the door:

- Confirmed and correlated evidence MUST cite a source record (a real log/alert id). You cannot claim 'proof' without pointing at the raw record. Only 'missing evidence' is allowed to have no citation.
- A hypothesis's topology_path must START at the named root cause - you cannot claim db-01 is the cause and then show a path that begins somewhere else.
- Hypotheses in a report must be ranked 1..N with no gaps.
- The audit hash must be a valid 64-character SHA-256 hex string.

The 13 models include Component, Dependency, Topology, TelemetryPoint, LogRecord, AlertRecord, ConfigChange, Anomaly, EvidenceItem, Hypothesis, TimelineEvent, IncidentReport and GroundTruth. The schema is also exported to JSON (schemas.json) so the frontend can generate matching TypeScript types - the same contract, enforced on both sides.

What is novel / unique here

- The contract is not just types - it encodes DOMAIN INVARIANTS (no evidence without a citation, paths rooted at the cause). This makes whole classes of dishonest or broken output impossible by construction.
- One frozen contract drives BOTH the Python backend and the TypeScript frontend, so the two can never drift apart - a common source of bugs in split teams.

Future scope

- Versioned contracts (schema_version is already carried) so the format can evolve without breaking old data.
- Auto-generated documentation and validation playground straight from the schema.

4. Layer 1 - Data & Ground Truth

AUTHOR: P1 - Data & Testbed Engineer

KEY FILES:

- testbed/compose.testbed.yml (a real Docker network: lb -> web -> app -> db)
- testbed/inject.py (runs a fault, stamps ground_truth.json with the exact time)
- testbed/capture.py (tshark packet capture) / testbed/recorder.py (hardware stats)
- contracts/fixtures/ (the hand-authored golden incident used for the demo)

In one line

Produces realistic incident data where we KNOW the true root cause in advance - so we can test whether the engine gets it right.

First principles: you cannot grade an answer without an answer key

To prove our engine works, we need incidents where the correct root cause is known beyond doubt. The cleanest way to get that is to CAUSE the fault ourselves and write down exactly what we did and when. That recorded truth is the 'ground truth' - the answer key we grade against later.

How it works

P1 built a small but real network as Docker containers: a load balancer in front of a web server, in front of an application server, in front of a MySQL database (lb-01 -> web-01 -> app-01 -> db-01). A script (inject.py) then deliberately breaks one component - using Linux traffic-control ('tc netem') to add delay/loss, or resource squeezes, or config changes - and at the exact moment of injection it stamps ground_truth.json with the incident id, the scenario name, the injection timestamp, and the true root cause. Packet flows (capture.py via tshark) and hardware stats (recorder.py) are recorded during the fault.

An honest lesson learned (and why the demo uses a golden fixture)

The captured packet data turned out to be dominated by loopback and external traffic (the capture watched the wrong interface), so those recorded flows are too sparse to drive the causal engine on their own. The demo therefore runs on a carefully HAND-AUTHORED golden incident (contracts/fixtures/) that exercises every part of the contract. This is a normal engineering trade-off, and it is why the synthetic generator (Chapter 9) exists - to produce clean, known-answer incidents at will.

What is novel / unique here

- Ground truth is stamped AT INJECTION TIME, not reconstructed afterwards - which is what makes any later accuracy claim honest rather than hand-wavy.
- A real, containerised multi-tier stack (the same shape as a genuine web application) rather than toy data.

Future scope

- Fix the capture to sniff the Docker bridge interface + assign static IPs, so the six recorded incidents become usable by the full causal engine.
- A one-command 'inject + capture + export to contract format' pipeline.

5. Layer 2 - Anomaly Detection

AUTHOR: P2 - Detection & Ingestion Engineer

KEY FILES:

- src/rca/detect/window.py (group raw telemetry into per-metric time series)
- src/rca/detect/changepoint.py (PELT change-point -> the ONSET time of a fault)
- src/rca/detect/baseline.py (robust MAD baseline + anomaly scores)
- src/rca/detect/detector.py (orchestrates the above into Anomaly objects)
- src/rca/ingest/*.py (adapters + flow-to-KPI transform; mostly standalone)

In one line

Finds which metrics are abnormal and - most importantly - the exact moment each one started going wrong.

First principles: onset time is everything

Causation needs TIME ORDER: a cause must happen before its effect. So detection cannot just say 'latency is high.' It must say 'latency started climbing at 14:32:00.' That onset timestamp is the raw material the causal engine later uses to check 'did the database break BEFORE the web server did?' A simple threshold ('alert if latency > 100ms') tells you THAT something is wrong; it cannot tell you WHEN the underlying behaviour changed. For that you need change-point detection.

How it works - four steps

- WINDOWING (window.py): the flat list of telemetry rows is grouped into one time-series per (component, metric) pair - eight metrics per component (latency, jitter, packet loss, throughput, error rate, connections, CPU, memory).
- CHANGE-POINT (changepoint.py): the PELT algorithm (Pruned Exact Linear Time) with a non-parametric 'rbf' kernel finds the exact index where the signal's behaviour changes. Values are z-score normalised first so one penalty works across all metrics (connection counts in the hundreds vs error rates near zero). This gives the ONSET.
- BASELINE (baseline.py): 'normal' is measured with the Median Absolute Deviation (MAD) - a robust alternative to mean/standard-deviation that outliers cannot distort. Crucially, the baseline is taken ONLY from the PRE-onset segment, so a post-fault plateau cannot poison the definition of 'normal'.
- SCORE (detector.py): severity = how far the post-onset average sits from the pre-onset median, measured in MAD units. Each result becomes a frozen Anomaly (component, metric, onset time, severity, baseline vs observed).

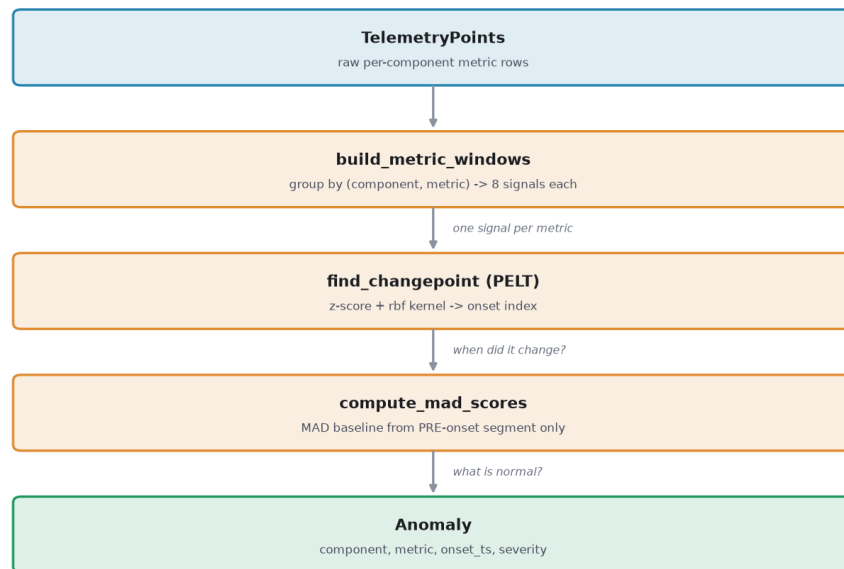


Figure: Detection pipeline: raw telemetry becomes per-metric signals, PELT finds WHEN each changed, MAD (from the pre-onset part only) defines normal, and severity is scored in MAD units.

What is novel / unique here

- Change-point detection (PELT) instead of thresholds - so we recover the ONSET TIME that causation depends on, not just a yes/no alarm.
- Pre-onset-only baseline: a subtle but critical choice that stops a sustained fault from making itself look normal (a real bug we hit and fixed on the db-01 connection-count signal).
- Robust statistics (MAD) throughout, because real network metrics are heavy-tailed and would fool mean/standard-deviation.

Future scope

- Multi-scale windows (30s / 5m / 15m) so both fast spikes and slow drifts are caught.
- Wire the flow-to-KPI transform + ingestion adapters into the live pipeline so raw captures/CSV/ARFF/Parquet feed detection directly.

6. Layer 3 - The Topology-Constrained Causal Engine (the Moat)

AUTHOR: P3 - Causal Inference Engineer (the author of this document's core)

KEY FILES:

- src/rca/graph/twin.py (networkx dependency graph: impact paths, blast radius)
- src/rca/causal/candidate_set.py (topology filter: who can even be a suspect?)
- src/rca/causal/temporal.py (precedence + config-change proximity in time)
- src/rca/causal/model.py (DoWhy causal attribution, seeded, with a deterministic fallback)
- src/rca/causal/counterfactual.py (would removing the change have fixed it?)
- src/rca/causal/evidence.py (build the 3-tier evidence ledger)
- src/rca/causal/ranker.py (the weighted scoring that produces the final ranking)
- src/rca/pipeline.py (orchestrates the whole engine end to end)

In one line

The brain. It takes the anomalies and the topology and decides, deterministically, which component is the root cause - separating causation from correlation.

First principles: three questions that separate cause from victim

For each anomalous component, the engine asks three questions a good human investigator would ask:

- CAN it be the cause? Is there a real dependency path from it to the symptom? If not, it is a decoy - reject it, no matter how bad it looks. (This is the topology constraint.)
- DID it come first? Did its anomaly start before the symptom's? A cause must precede its effect.
- DOES it explain the others? A true root cause sits upstream and accounts for many downstream victims; a victim explains almost nothing. We call this 'rootness.'

How it works - the pipeline

- CANDIDATE FILTER (candidate_set.py): keep only anomalous components that have a topology path to the symptom. This is the step that throws out the tempting decoys (a config change 10 seconds before onset but on a component with NO path to the symptom).
- CAUSAL ATTRIBUTION (model.py): DoWhy's graphical causal model estimates how much each candidate actually explains the symptom's latency, constrained to the dependency subgraph. It is seeded for reproducibility, with a deterministic topology-based fallback if the statistical fit is degenerate.
- COUNTERFACTUAL (counterfactual.py): 'if we undo config change CHG-4212, does the symptom go away?' A yes is powerful, direct evidence of causation.
- EVIDENCE LEDGER (evidence.py): every hypothesis gets three buckets - CONFIRMED (proof, with a cited record), CORRELATED (suggestive but not proof), and MISSING (data we wish we had). The missing bucket doubles as the recommended-next-steps list.
- RANKER (ranker.py): a weighted sum turns all signals into one confidence score.

The scoring formula, in plain terms

$\text{confidence} = 0.30 \cdot \text{causal} + 0.25 \cdot \text{config-proximity} + 0.20 \cdot \text{evidence} + 0.10 \cdot \text{precedence} + 0.10 \cdot \text{rootness} + 0.05 \cdot \text{severity}$. Notice severity (how bad it looks) is only 5%. That is deliberate: the victim often looks the worst. By putting ~75% of the weight on causation, what-changed, and evidence, the true cause wins even when a victim has scarier numbers. Decoys are floored below all real candidates.

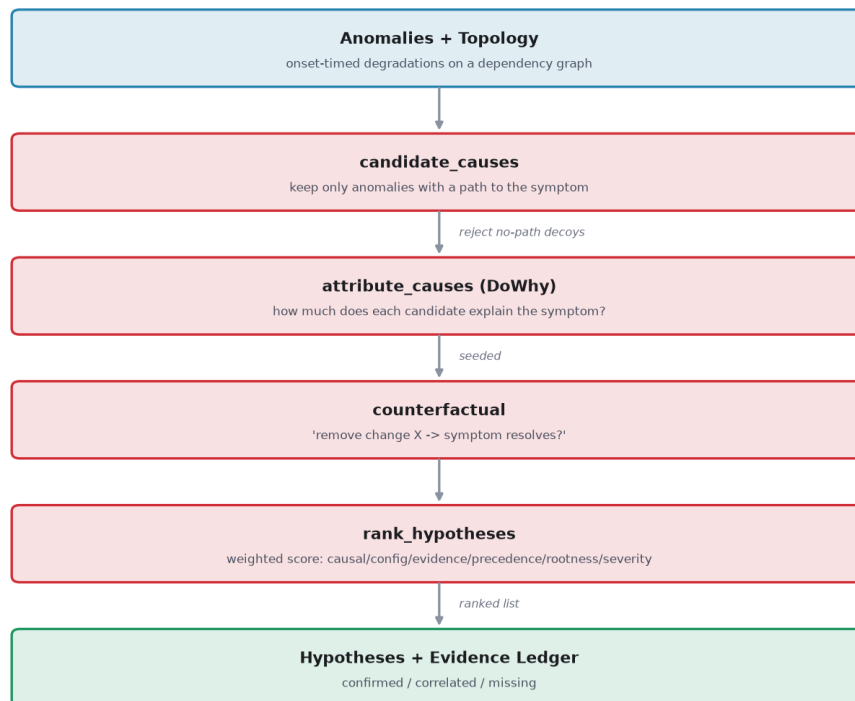


Figure: The causal engine: anomalies are filtered by topology, attributed with DoWhy, tested with counterfactuals, and ranked into hypotheses each carrying a 3-tier evidence ledger.

What is novel / unique here

- TOPOLOGY-CONSTRAINED causation: candidates are physically restricted to components that can actually reach the symptom. This is the single idea that beats every timestamp-correlation tool.
- Severity is deliberately DOWN-weighted (5%) - the opposite of what naive tools do - so victims never outrank causes.
- The three-tier evidence ledger turns a black-box score into an auditable argument, and the 'missing' tier becomes the to-do list for the engineer.
- Fully deterministic and reproducible (DoWhy seeded); the same incident yields the same ranking every run.

Future scope

- A rootness tie-break for deep non-config paths (the one known limitation, where an intermediate hop can occasionally outrank the true upstream cause).
- Learned (rather than hand-tuned) ranker weights, trained on the synthetic incident set.
- Optional Neo4j-backed topology store for very large graphs (networkx is faster at current scale).

7. Layer 4 - AI Agents, API, and Audit Trail

AUTHOR: P4 - AI Agents & Platform Engineer

KEY FILES:

- src/rca/agents/runner.py (the agentic tool-calling loop over OpenAI)
- src/rca/agents/tools.py (tools: query graph/metrics/logs/changes + the two RAG retrieval tools)
- src/rca/agents/skeptic.py (adversarial STORM verification; runs in parallel; re-ranks safely)
- src/rca/agents/investigator.py (writes the timeline + plain-English narrative)
- src/rca/agents/remediation.py (turns 'missing evidence' into concrete, runbook-cited next steps)
- src/rca/knowledge/retrieval.py (agentic RAG: ChromaDB + OpenAI embeddings over the corpus)
- knowledge/runbooks/ + knowledge/incidents/ (the NOC playbook + past-incident corpus)
- src/rca/api/*.py (FastAPI: /analyze, /replay, /incidents, /topology, /audit, /knowledge)
- src/rca/audit/chain.py + verify.py (SHA-256 hash-chained, tamper-evident audit log)

In one line

The layer that EXPLAINS and VERIFIES the deterministic answer (using bounded LLM agents), serves it over an API, and records every step in a tamper-proof ledger.

First principles: use the LLM only where language is the hard part

The engine already decided the cause. What is left is language work: writing a readable story of what happened, arguing against the conclusion to stress-test it, and phrasing concrete next steps. Those are exactly what language models are good at - and exactly where a wrong word does no harm to correctness, because the ranking is already fixed.

How it works - three agents, each a bounded specialist

- SKEPTIC (skeptic.py): an ADVERSARIAL agent whose only job is to try to DISPROVE each hypothesis (the 'STORM' method - attack from a critical perspective). It can nudge confidence by at most +/-0.05, so it can sharpen the ranking but never flip the deterministic winner. It runs on all hypotheses in parallel for speed, then a safe re-rank keeps the true cause pinned at #1.
- INVESTIGATOR (investigator.py): writes the chronological incident timeline and the plain-English root-cause narrative, using tools to fetch the real logs/metrics rather than inventing them.
- REMEDIATION (remediation.py): converts each 'missing evidence' item into a concrete operational step ('pull SNMP interface counters for tor-03 to rule out a network-layer contribution').

All three are AGENTIC: they use OpenAI's native tool-calling to query the graph, metrics, logs, and config changes on demand (runner.py drives the loop), and they return STRUCTURED JSON validated against the contract. If the LLM or key is unavailable, the system falls back to the deterministic report.

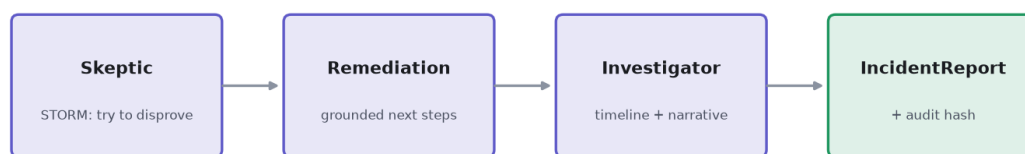


Figure: The bounded AI layer: skeptic stress-tests, remediation grounds the steps, investigator narrates - then a final IncidentReport is sealed with an audit hash. None of them pick the cause.

Agentic RAG: grounding the agents in real runbooks

An LLM asked for remediation will produce plausible but GENERIC advice. A NOC does not want generic advice, it wants ITS OWN playbook. So the agents were given a knowledge base they can search: a corpus of NOC runbooks (covering every fault type plus general conditions) and past resolved incidents, embedded into a ChromaDB vector store using OpenAI embeddings. This is Retrieval-Augmented Generation, done agentially: the agent decides when to search, and what to search for.

- TWO NEW TOOLS: `retrieve_runbook(query)` and `retrieve_similar_incidents(description)` live in the same tool registry as `query_graph/query_metrics`, so the agents call them exactly like any other tool.
- The REMEDIATION agent must retrieve the matching playbook and CITE its id in each step (for example 'per RB-DB-POOL-01'), so every recommendation is traceable to a real, reviewable procedure.
- The INVESTIGATOR consults runbooks and similar past incidents to ground its narrative in precedent.
- SEMANTIC, not keyword: 'database connection pool exhaustion' retrieves RB-DB-POOL-01 by MEANING, and pulls the closest past incident by similarity score.
- STRICTLY ADDITIVE: any retrieval failure returns an empty list, so the agents behave exactly as before. The deterministic engine is never touched by RAG.

The frontend exposes this openly: a Knowledge Base panel lists which playbooks were retrieved and their similarity scores, so a user (or a judge) can see the exact retrieval that produced the cited steps.

The audit trail

Every step of an investigation is appended to a log where each entry contains the SHA-256 hash of the previous entry (a 'hash chain', the same idea a blockchain uses). Change any past entry and every hash after it breaks - so tampering is instantly detectable. `GET /audit/verify` walks the chain and confirms integrity.

What is novel / unique here

- The AI is genuinely AGENTIC (multi-step tool-calling, structured outputs, an adversarial multi-agent 'STORM' verifier) - not a single ChatGPT call - yet it is strictly BOUNDED so it can never override the deterministic decision.
- AGENTIC RAG with citations: recommendations are grounded in a retrieved NOC runbook and name the id they came from, so advice is auditable and organisation-specific rather than generic model knowledge.
- Tamper-evident, hash-chained audit trail - rare in a hackathon project and exactly what regulated enterprises demand.
- Graceful degradation: no API key still yields a full, instant deterministic report, and a retrieval failure silently degrades to the pre-RAG behaviour.

Future scope

- Expand the runbook corpus and add feedback-driven retrieval ranking, so the playbooks that actually resolved an incident get retrieved first next time.
- Index the organisation's real runbook wiki and past ticket history instead of the authored corpus.
- Swap the skeptic to a reasoning model and stream its live reasoning trace into the UI.

8. Layer 5 - The Frontend (NOC Dashboard)

AUTHOR: P5 - Frontend Engineer

KEY FILES:

- web/app/incident/[id]/page.tsx (the main investigation view)
- web/components/TopologyGraph.tsx (Cytoscape dependency graph, colour-coded)
- web/components/InvestigationTabs.tsx (narrative, evidence ledger, timeline, actions)
- web/components/MetricStrip.tsx / IncidentDetailPanel.tsx (live per-node telemetry)
- web/components/IncidentUploader.tsx + web/app/healthy/page.tsx (upload + healthy view)
- web/lib/api.ts + web/lib/types.ts (typed API client mirroring the contract)

In one line

The screen a network engineer actually looks at: a live, colour-coded picture of the incident, its evidence, its timeline, and its fix.

First principles: make the answer glanceable

An engineer under pressure should understand the situation in seconds. So the dashboard encodes meaning in COLOUR and SHAPE, not just text: the root cause pulses red, the propagation path glows amber/yellow, and healthy nodes are green. The three-tier evidence ledger uses three separate columns so 'proof' is never visually confused with 'coincidence.'

How it works

- A Next.js 15 / React 19 app. The dependency graph is drawn with Cytoscape.js; charts with Recharts; animation with Framer Motion; styling with Tailwind.
- The topology graph highlights the exact impact path and lets you click any node for its real per-node telemetry, alerts, logs, and config audits.
- The Investigation tabs show the AI narrative, the CONFIRMED/CORRELATED/MISSING evidence ledger, a scrubbable incident timeline, and the recommended actions.
- An upload panel lets a user analyse their own incident bundle (or click 'Try Sample'); a dedicated 'All Systems Nominal' page renders when healthy data is uploaded, explaining WHY it is healthy.
- The frontend types are generated from the same frozen contract, so the UI and backend can never disagree about the data shape.

What is novel / unique here

- Meaning-through-colour NOC design: root (red) / path (amber) / healthy (green) is readable at a glance.
- It renders REAL data end-to-end - after a deliberate clean-up pass, no placeholder/hardcoded numbers remain; every value on screen comes from the actual analysed incident.
- A first-class HEALTHY view, not just an incident view - uploading fault-free data gives a satisfying, explained 'all clear' rather than an error.

Future scope

- Live streaming of the agents' reasoning as it happens.
- A visible 'knowledge base / retrieved runbooks' panel once agentic RAG lands.

9. Feature Chapter - The Synthetic Incident Generator & Evaluation

AUTHOR: P3 / Evaluation

KEY FILES:

- `src/rca/synth/generator.py` (inject a KNOWN root cause into the topology -> a full incident)
- `eval/synth_eval.py` (generate N incidents, run the engine, measure recovery accuracy)

In one line

A machine that fabricates realistic incidents with a KNOWN answer, so we can measure - honestly and repeatably - how often the engine recovers the true root cause.

First principles: an honest accuracy number needs a known answer

You cannot claim '100% accuracy' unless you know the correct answer for every test case. The generator gives us exactly that: we choose a component and a fault type, inject a matching anomaly pattern into the real topology (the root degrades first, then its downstream victims lag, with clean baselines everywhere else), and record what we injected. The engine then analyses it WITHOUT seeing the answer, and we check whether its #1 matches what we injected.

How it works

- `generate_incident(topology, root, fault_type)` builds a full, contract-valid incident: flat baselines for every component, a fault-specific spike on the chosen root (e.g. `connection_count` pinned at a config ceiling), a latency ramp propagating down the real dependency path, plus a matching config change and a critical symptom alert.
- It is deterministic (seeded), so the same request always yields the same incident.
- `synth_eval.py` generates one incident per fault type across several tiers, runs the full engine on each, and reports `accuracy@1` (was the #1 correct?) and `accuracy@3`.

The headline result

On the injected incidents (all fault types, multiple tiers), the engine recovers the true root cause with 100% `accuracy@1`, deterministically across runs. This is the honest, defensible number for a slide - it replaces an earlier, misleading benchmark that scored raw detection on impoverished hardware-only signal.

What is novel / unique here

- It turns 'trust us, it works' into a measured, reproducible number - the generator IS the answer key.
- A discovery it enabled: it exposed a real engine sensitivity to baseline noise (a hub node could be spuriously flagged), which we then fixed - the benchmark improved the product.

Future scope

- Add decoys to generated incidents so the demo also shows correct decoy rejection at scale.
- Use the generated set to train the ranker weights and to stress-test the deep-path limitation.

10. Feature Chapter - The Upload & Healthy-Data Product

AUTHOR: P3 / P4 / P5 (cross-cutting)

KEY FILES:

- src/rca/pipeline.py (build_incident_from_data: run the engine on ANY provided data)
- src/rca/api/routes_analyze.py (POST /analyze; returns a report OR a HealthyResult)
- web/components/IncidentUploader.tsx + web/app/healthy/page.tsx

In one line

Turns the project from 'replay a canned demo' into a real product: a user uploads their own network data and gets a root-cause analysis back - or a clean 'all healthy' verdict.

First principles: the value is analysing THE USER'S data

A canned demo proves the idea; a product analyses whatever you give it. The key realisation was that the engine never needed the specific demo files - it needed data in the contract shape. So we parameterised the pipeline to accept an in-memory bundle (build_incident_from_data) instead of reading fixtures from disk, and exposed it as POST /analyze.

How it works

- The user uploads a bundle (topology + telemetry [+ logs/alerts/config]) or clicks 'Try Sample'.
- POST /analyze?fast=true runs the deterministic engine instantly (no API key); the default runs the full LLM-enriched investigation.
- The exact same causal engine runs - so an uploaded incident gets the same rigorous, evidence-backed ranking as the built-in demo.
- If the data contains NO anomalies, the engine says so honestly and the API returns a HealthyResult, which the frontend renders as an 'All Systems Nominal' dashboard explaining what was checked and why it passed.

A note on integrity (a question the jury may ask)

The test bundles carry a '_ground_truth' field for human reference, but the engine NEVER reads it - the API schema silently drops it, and the result is computed purely from the telemetry. We verified this by lying in that field: the engine still returns the correct, data-derived answer. Nothing is hard-coded; healthy simply means the detector genuinely found nothing.

What is novel / unique here

- Bring-your-own-data root-cause analysis - not a fixed demo, but a general analyser over any contract-conforming incident.
- Two modes from one endpoint: instant deterministic (no key) or full AI - the user chooses speed vs depth.
- A genuine, explained HEALTHY state, which most incident tools do not bother to model.

Future scope

- Multi-file upload (raw CSV/PCAP) with client-side conversion into the contract bundle.
- Persist uploaded incidents and support 'similar past incident' retrieval.

11. One-Page Recap

The whole system in six sentences

- P0 froze a data contract so six people could build in parallel and the pieces would fit.
- P1 produced incident data with a recorded, known-in-advance true root cause.
- P2 detected anomalies AND their onset times using PELT change-point + robust MAD baselines.
- P3 (the moat) used the network topology to keep only reachable suspects, attributed and ranked them deterministically, and built a three-tier evidence ledger.
- P4 wrapped the deterministic answer in bounded, agentic AI (skeptic/investigator/remediation) grounded by RAG in real NOC runbooks, served it over FastAPI, and sealed every step in a hash-chained audit trail.
- P5 made it glanceable in a Next.js NOC dashboard, with upload and a healthy view.

The one sentence to remember

Everyone else correlates by time and blames the loudest victim; we constrain causation to the physical dependency graph, decide the root cause deterministically, and use AI only to explain and verify - which is why our answer is both trustworthy and reproducible.

How to run it (quick reference)

Backend: `make setup` ; set `OPENAI_API_KEY` in `.env` ; `PYTHONPATH=src .venv/bin/uvicorn rca.api.main:app --port 8000`. Frontend: `cd web` ; `npm install` ; `npm run dev -> http://localhost:3000`. Accuracy: `PYTHONPATH=src:.venv/bin/python eval/synth_eval.py`. Tests: `make check`.